

AN INTERNSHIP REPORT ON
EV BATTERY MANAGEMENT SYSTEM

*Submitted in partial fulfillment of the requirements for the completion of the
Virtual Internship Programme*

Submitted by
Vijaya Pradhap S V

Internship Organization
Elevance Skills

Project Duration:
May 2026 - July 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Elevance Skills** for providing me with the opportunity to undertake this virtual internship and successfully complete the **EV Battery Management System** project.

This internship provided valuable hands-on experience in embedded systems, IoT, firmware development, and battery management system design. It enabled me to strengthen my practical knowledge by applying theoretical concepts using the ESP32 microcontroller, PlatformIO, Wokwi simulation, and the Blynk IoT platform.

I sincerely appreciate the learning resources, project guidelines, and support provided throughout the internship, which contributed significantly to the successful completion of this project.

Finally, I would like to thank my family, friends, and peers for their constant encouragement, motivation, and support throughout the internship period.

Vijaya Pradhap S V

ABSTRACT

Battery Management Systems (BMS) play a vital role in ensuring the safe, reliable, and efficient operation of rechargeable battery packs used in electric vehicles, renewable energy storage systems, and portable electronic devices. This report presents the design, development, and simulation-based validation of an **EV Battery Management System** developed using the ESP32 microcontroller. The system is capable of monitoring individual cell voltages, calculating pack-level parameters, estimating State of Charge (SOC) and State of Health (SOH), and providing protection against over-voltage, under-voltage, and over-temperature conditions.

The system was developed using the PlatformIO ecosystem in Visual Studio Code with the Arduino framework. Four potentiometers were used to emulate the voltage characteristics of four battery cells, enabling safe and repeatable testing of the protection logic without the need for an actual battery pack. A 16×2 I2C LCD displays real-time battery parameters and system status, while a relay module and buzzer provide protection by disconnecting the load and generating an audible alert during fault conditions. The Wokwi simulator was used extensively for simulation and validation, and the Blynk IoT platform was integrated to enable remote monitoring of battery parameters through a mobile dashboard.

The firmware follows a modular software architecture consisting of Battery, Safety, Display, Runtime, Telemetry, Configuration, and Utility modules, improving code maintainability, scalability, and ease of testing. Experimental results demonstrate reliable fault detection, consistent relay operation, accurate monitoring of battery parameters, and stable cloud-based telemetry. The developed system provides a practical and cost-effective prototype that can serve as a foundation for future EV battery monitoring and protection systems.

Keywords: Battery Management System (BMS), EV Battery Management System, ESP32, State of Charge (SOC), State of Health (SOH), Embedded Systems, IoT, Blynk, Wokwi, Battery Protection.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	2
ABSTRACT	3
TABLE OF CONTENTS	4
1. INTRODUCTION	1
2. OBJECTIVES	2
3. SYSTEM DESIGN	3
3.1 Design Philosophy	3
3.2 High-Level System Architecture	3
3.3 Functional Block Diagram	4
3.4 Data Flow Description	5
3.5 Design Parameters and Constraints	6
4. HARDWARE IMPLEMENTATION	7
4.1 ESP32 DevKit	7
4.2 16×2 LCD with I2C Interface	7
4.3 Relay Module	7
4.4 Buzzer	7
4.5 Potentiometers (Battery Cell Emulation)	8
4.6 Component Summary	8
4.7 Power Supply Considerations	9
4.8 Pin Assignment	9
5. SOFTWARE ARCHITECTURE	11
5.1 Module Overview	11
5.2 Battery Module	11
5.3 Safety Module	12
5.4 Display Module	12
5.5 Runtime Module	12
5.6 Telemetry Module	12
5.7 Configuration Module	12
5.8 Utility Module	13

5.9 Build and Development Environment	13
6. METHODOLOGY	14
6.1 Requirement Analysis	14
6.2 Hardware Selection and Circuit Design	14
6.3 Firmware Development	14
6.4 Simulation-Based Verification	14
6.5 Cloud Integration and Telemetry Testing	15
6.6 Test Case Design	15
7. RESULTS AND DISCUSSION	16
7.1 Functional Verification Results	16
7.2 Response Time of Protection Mechanism	16
7.3 SOC and SOH Estimation Behaviour	17
7.4 Telemetry Performance	17
7.5 Discussion	18
8. ADVANTAGES & APPLICATIONS	19
8.1 Electric Mobility	19
8.2 Renewable Energy Storage	19
8.3 Unmanned Aerial Vehicles and Robotics	20
8.4 Portable and Backup Power Systems	20
8.5 Educational and Research Platforms	20
9. FUTURE SCOPE	21
10. CONCLUSION	22
11. REFERENCES	23

1. INTRODUCTION

The increasing use of lithium-ion and lithium-polymer batteries in electric vehicles, renewable energy storage systems, and portable electronic devices has made battery monitoring and protection an essential requirement. These batteries are highly sensitive to overcharging, over-discharging, and excessive temperature, which can reduce battery life and, in severe cases, lead to safety hazards. A Battery Management System (BMS) continuously monitors battery parameters and ensures safe operation by protecting the battery from abnormal operating conditions.

This project presents the design and implementation of an **EV Battery Management System** using the **ESP32** microcontroller. The ESP32 was selected because of its built-in Wi-Fi and Bluetooth connectivity, multiple analog-to-digital converter (ADC) channels, and compatibility with the Arduino framework and PlatformIO development environment, making it suitable for embedded IoT applications.

To provide a safe and cost-effective development environment, four potentiometers were used to emulate the voltages of four battery cells connected in series. The system continuously monitors individual cell voltages, calculates the total pack voltage, estimates the **State of Charge (SOC)** and **State of Health (SOH)**, and provides protection against abnormal conditions using a relay and buzzer. A **16×2 I2C LCD** displays battery information locally, while the **Blynk IoT** platform enables remote monitoring through a mobile application. The firmware was developed and validated using the Wokwi simulation environment.

This report describes the complete development of the project, including the system design, hardware implementation, software architecture, development methodology, testing, and performance evaluation. The project demonstrates the practical application of embedded systems and IoT technologies in developing a low-cost prototype for battery monitoring and protection.

2. OBJECTIVES

The primary objective of this internship project is to design, implement, and validate an IoT-enabled EV Battery Management System using the ESP32 microcontroller. The specific objectives are as follows:

1. To design an ESP32-based Battery Management System capable of monitoring the voltages of four simulated battery cells connected in series.
2. To calculate key battery parameters, including individual cell voltage, total pack voltage, State of Charge (SOC), and State of Health (SOH).
3. To implement battery protection against over-voltage and under-voltage conditions using a relay and buzzer.
4. To display real-time battery information and system status on a 16×2 I2C LCD.
5. To integrate the system with the Blynk IoT platform for remote monitoring of battery parameters.
6. To develop the firmware using a modular software architecture for improved readability, maintainability, and scalability.
7. To validate the complete system using the Wokwi simulation environment under normal and fault conditions.
8. To document the design, implementation, testing, and performance of the developed system.

3. SYSTEM DESIGN

3.1 Design Philosophy

The system was designed around three guiding principles: safety-first operation, modularity, and observability. Safety-first operation means that protective actions, such as relay disconnection, take precedence over all other system functions and are evaluated on every control loop iteration regardless of the state of the display or telemetry subsystems. Modularity ensures that each functional concern, such as sensing, safety evaluation, or telemetry transmission, is isolated within its own software module with a well-defined interface, which simplifies testing and future extension. Observability ensures that the system status and battery parameters can be monitored locally through the LCD and remotely through the Blynk dashboard.

3.2 High-Level System Architecture

At a high level, the system consists of a sensing layer, a processing layer, and an output layer. The sensing layer comprises the four potentiometers representing battery cells. The processing layer is implemented entirely on the ESP32 and is responsible for reading raw analog values, converting them into calibrated voltages, computing derived parameters such as SOC, SOH, and runtime, and evaluating safety conditions. The output layer comprises the LCD for local display, the relay and buzzer for physical protective action, and the Wi-Fi-based Blynk telemetry link for remote monitoring.

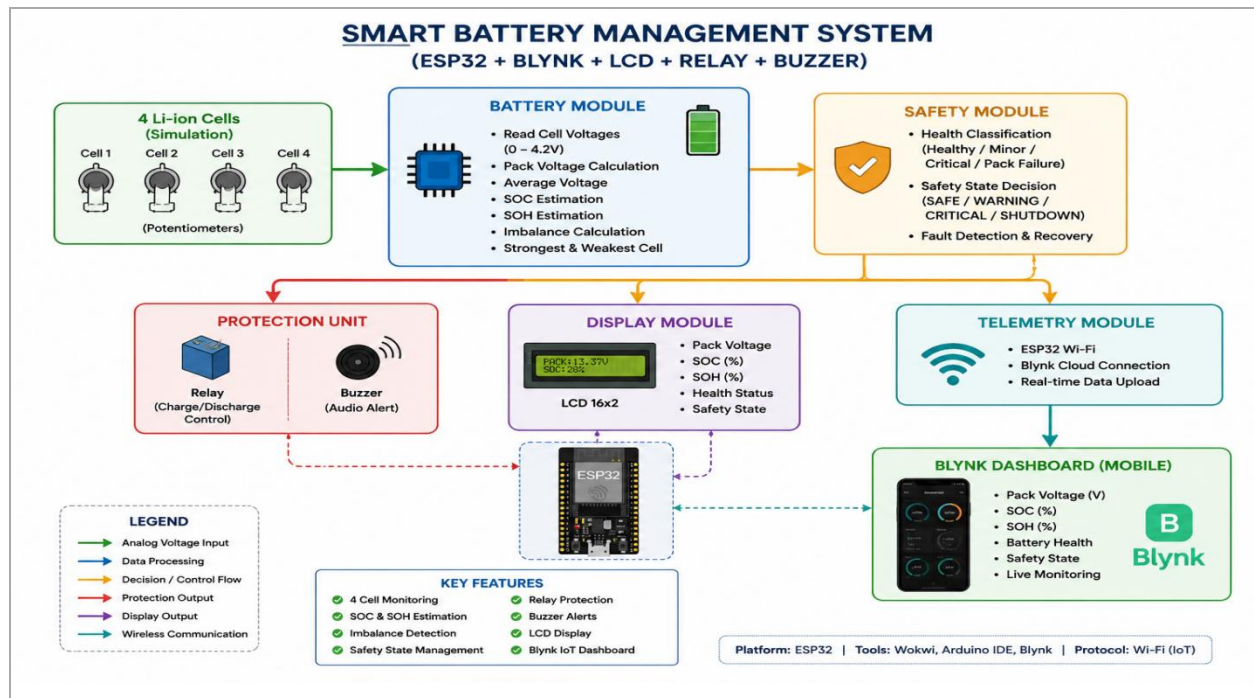


Figure 3.1: High-Level System Architecture of the EV Battery Management System

3.3 Functional Block Diagram

The functional block diagram below illustrates the flow of information from the sensing elements through the ESP32's processing pipeline to the various output subsystems. Each block corresponds to a distinct responsibility within the firmware's modular architecture.

SMART BATTERY MANAGEMENT SYSTEM

WORKFLOW DIAGRAM

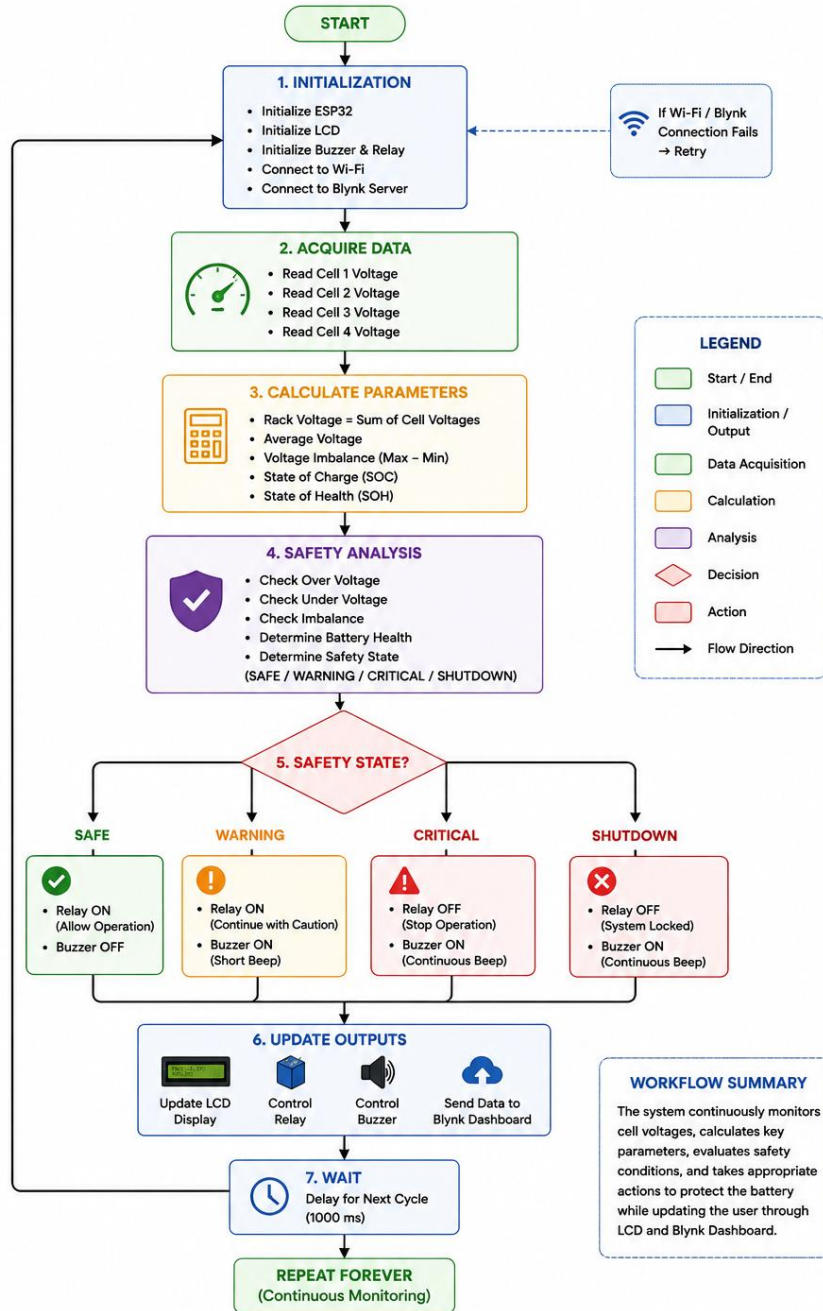


Figure 3.2: Functional Block Diagram Showing Data Flow Between Modules

3.4 Data Flow Description

On every iteration of the main control loop, the Battery Module reads the four analog channels corresponding to the simulated cell voltages and applies a calibration scale factor to convert raw ADC counts into calibrated voltage values. These calibrated voltages are aggregated into a pack voltage and passed to the estimation routines that compute SOC, SOH, and estimated runtime. The Safety Module then evaluates the calibrated voltages against the configured safety thresholds. If any threshold is violated, it generates a fault condition that activates the relay protection and buzzer while updating the LCD and Blynk dashboard.

3.5 Design Parameters and Constraints

The following table summarises the key design parameters and operating constraints adopted for the system.

Parameter	Value / Range	Remarks
Number of simulated cells	4	Series configuration
Nominal cell voltage	3.7 V	Typical Li-ion nominal value
Cell over-voltage threshold	4.2 V	Per-cell protection limit
Cell under-voltage threshold	3.0 V	Per-cell protection limit
Nominal pack voltage	14.8 V	4S configuration
ADC resolution	12-bit	ESP32 native ADC
Sampling interval	500 ms	Main control loop period
Telemetry update interval	2 s	Configurable via Configuration Module

Table 3.1: Key Design Parameters and Operating Constraints

4. HARDWARE IMPLEMENTATION

This chapter describes the hardware components used to implement the EV Battery Management System and explains the function of each component within the overall system.

4.1 ESP32 DevKit

The ESP32 DevKit serves as the central processing and communication unit of the system. It provides multiple ADC channels, digital GPIO pins, I2C communication, and built-in Wi-Fi connectivity. In this project, it acquires battery cell voltages, processes battery parameters, controls the relay and buzzer, drives the LCD, and transmits telemetry to the Blynk cloud.

4.2 16×2 LCD with I2C Interface

A 16-character, 2-line liquid crystal display fitted with an I2C backpack module (typically based on the PCF8574 I/O expander) is used to provide local, at-a-glance visibility of system status without requiring a network connection. The I2C interface reduces the wiring overhead from the eight or more parallel data lines required by a standard HD44780-compatible LCD down to just two signal lines (SDA and SCL), which simplifies the circuit and frees up GPIO pins for other peripherals.

4.3 Relay Module

A single-channel relay module is used as the physical disconnect mechanism for the battery pack. The relay coil is driven through a GPIO pin via the module's onboard transistor driver circuit, allowing the low-current ESP32 output to switch the higher-current contacts that would, in a physical deployment, be wired in series with the simulated battery circuit's power path. During normal operation the relay remains energised (closed), and it is de-energised (opened) immediately upon detection of a fault condition.

4.4 Buzzer

An active buzzer provides an immediate, human-audible indication of fault conditions. It is driven directly from a digital GPIO output and is activated concurrently with the relay

disconnection sequence, ensuring that personnel in the vicinity of the system are alerted even if they are not actively observing the LCD or the Blynk dashboard.

4.5 Potentiometers (Battery Cell Emulation)

Four linear potentiometers are used to emulate the cell voltage behaviour of four individual battery cells connected in series. Each potentiometer's wiper is connected to one of the ESP32's ADC-capable input pins, and its resistance is manually adjusted during testing to reproduce normal, over-voltage, and under-voltage conditions in a controlled and repeatable manner. This substitution avoids the safety hazards and equipment cost associated with deliberately over-charging or over-discharging real lithium-ion cells during development and testing.

4.6 Component Summary

Component	Quantity	Function	Interface
ESP32 DevKit	1	Main processing and connectivity unit	—
16×2 LCD (I2C)	1	Local real-time information display	I2C (SDA/SCL)
Relay Module	1	Battery pack disconnect on fault	Digital GPIO
Buzzer	1	Audible fault alarm	Digital GPIO
Potentiometer	4	Simulated battery cell voltage source	Analog (ADC)
Jumper Wires / Breadboard	As required	Circuit interconnection	—

Table 4.1: Summary of Hardware Components Used

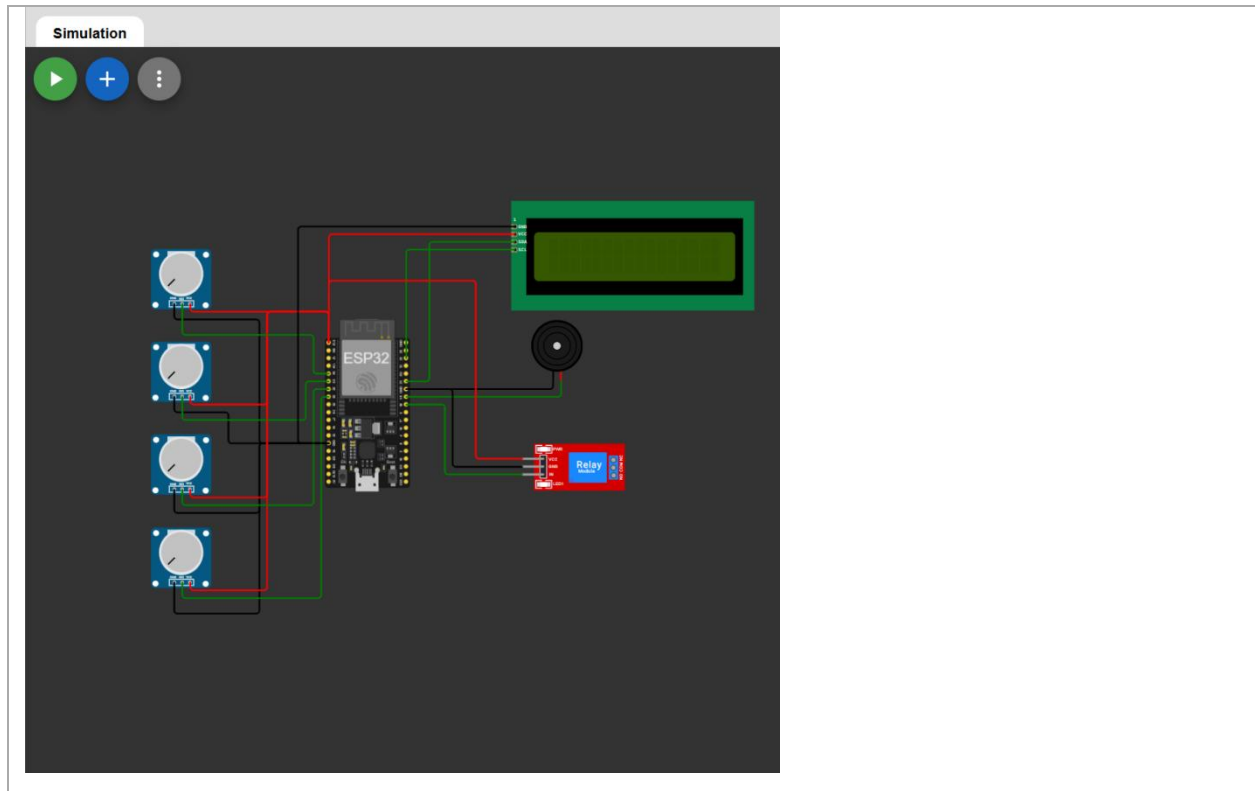


Figure 4.1: Wokwi Simulation Model of the EV Battery Management System

4.7 Power Supply Considerations

During development and simulation, the ESP32 DevKit and associated peripherals were powered via USB from the host computer, which is sufficient for the low-current LCD, buzzer, and relay coil driver circuit used in this prototype. In a production deployment intended to manage a real battery pack, the relay contacts — rather than the ESP32's own supply — would be wired in series with the actual high-current battery path, while the ESP32 and its peripherals would be powered from a regulated low-power tap, isolated from the switched high-current path.

4.8 Pin Assignment

ESP32 Pin	Connected To	Signal Type
GPIO 34	Potentiometer 1 (Cell 1) wiper	Analog Input
GPIO 35	Potentiometer 2 (Cell 2) wiper	Analog Input
GPIO 32	Potentiometer 3 (Cell 3) wiper	Analog Input
GPIO 33	Potentiometer 4 (Cell 4) wiper	Analog Input

ESP32 Pin	Connected To	Signal Type
GPIO 21	LCD I2C – SDA	I2C Data
GPIO 22	LCD I2C – SCL	I2C Clock
GPIO 18	Relay Module – IN	Digital Output
GPIO 19	Buzzer – Positive Terminal	Digital Output
3V3 / GND	Common supply rail for all peripherals	Power

Table 4.2: ESP32 Pin Assignment Table

5. SOFTWARE ARCHITECTURE

The firmware for the EV Battery Management System is organized as a collection of modular, loosely coupled components, each responsible for a specific functional task. This modular architecture improves code readability, simplifies debugging and testing, and allows individual modules to be modified or extended without affecting the rest of the firmware.

5.1 Module Overview

Module	Responsibility
Battery Module	Reads raw ADC values, applies calibration, computes cell and pack voltages
Safety Module	Evaluates voltage readings against safety thresholds and raises fault flags.
Display Module	Formats and renders system parameters on the 16×2 I2C LCD
Runtime Module	Coordinates execution of all other modules within the main control loop
Telemetry Module	Packages system parameters and transmits them to the Blynk IoT platform
Configuration Module	Centralises tunable constants such as thresholds, pin assignments, and intervals
Utility Module	Provides shared helper functions, e.g. value mapping, filtering, and formatting

Table 5.1: Firmware Module Overview

5.2 Battery Module

The Battery Module is responsible for interfacing with the four potentiometer-based analog inputs. It performs periodic ADC sampling, applies a light digital smoothing filter to reduce

reading noise, and converts the raw counts into calibrated voltage values using scale factors defined in the Configuration Module. It exposes the individual cell voltages, the summed pack voltage, and the average cell voltage to the rest of the system.

5.3 Safety Module

The Safety Module consumes the calibrated battery parameters produced by the Battery Module and compares them against the over-voltage and under-voltage thresholds defined in the Configuration Module. If any threshold is breached, it sets a system-wide fault flag and identifies the specific fault type, which downstream modules use to drive the relay, buzzer, LCD warning message, and telemetry fault indicator.

5.4 Display Module

The Display Module manages all interaction with the 16×2 I2C LCD, including screen initialisation, cursor positioning, and periodic refresh of pack voltage, SOC, and status text. To avoid visible flicker, the module only rewrites LCD segments whose values have changed since the previous refresh cycle.

5.5 Runtime Module

The Runtime Module acts as the orchestrator of the application. It owns the main control loop, invoking the Battery, Safety, Display, and Telemetry modules in a fixed, deterministic sequence on each cycle, and manages the timing intervals that govern how frequently each subsystem is serviced. This centralised coordination avoids race conditions between modules and keeps the overall control flow easy to reason about.

5.6 Telemetry Module

The Telemetry Module manages the Wi-Fi connection to the Blynk cloud and maps internal firmware variables, such as pack voltage, SOC, SOH, runtime estimate, and fault status, onto Blynk virtual pins at a configurable update interval. It also handles reconnection logic in the event of a temporary loss of Wi-Fi connectivity.

5.7 Configuration Module

The Configuration Module centralises all tunable constants used throughout the firmware, including GPIO pin assignments, voltage calibration factors, safety thresholds, sampling intervals, and Blynk authentication credentials. Centralising these values in a single header file avoids magic numbers scattered throughout the codebase and simplifies recalibration or threshold adjustment during testing.

5.8 Utility Module

The Utility Module provides small, reusable helper functions used across multiple other modules, such as value mapping and range-clamping functions, simple moving-average filters for ADC smoothing, and string formatting helpers for the LCD and serial debug output.

5.9 Build and Development Environment

The firmware was developed using **Visual Studio Code** with the **PlatformIO** extension, targeting the **Arduino framework** for the ESP32 platform. PlatformIO provided project management, library dependency handling, firmware compilation, uploading, and serial monitoring within a single integrated development environment. Version control was maintained using **Git**, with the project hosted on **GitHub** to manage source code revisions and track project development.

Tool	Purpose
PlatformIO	Build system, library management, and firmware upload
Visual Studio Code	Primary code editor and development environment
Arduino Framework	Hardware abstraction layer for ESP32 peripherals
Wokwi Simulator	Hardware-in-the-loop style firmware simulation and debugging
Blynk IoT	Cloud telemetry platform and mobile dashboard
GitHub	Source code version control and project hosting

Table 5.2: Software Tools Used in Development

6. METHODOLOGY

The project was carried out in a series of structured phases, moving from requirement definition through to final validation. This chapter describes the methodology followed at each stage of development.

6.1 Requirement Analysis

The first phase involved identifying the functional requirements of the system, namely cell voltage monitoring, pack-level parameter computation, SOC/SOH estimation, safety protection, local display, and cloud telemetry, and translating these into measurable design targets such as the voltage thresholds and sampling rates summarised in Chapter 4.

6.2 Hardware Selection and Circuit Design

Based on the identified requirements, the hardware components listed in Chapter 5 were selected, and a circuit schematic was drafted showing the interconnection between the ESP32, the LCD, the relay module, the buzzer, and the four potentiometers. Particular attention was paid to correct voltage-divider scaling on the potentiometer outputs to keep signals within the ESP32 ADC's safe input range.

6.3 Firmware Development

Firmware development followed an incremental approach. Development began with the Battery Module and basic serial-monitor output, followed by the Display Module, then the Safety Module, and finally the Telemetry Module, with the Configuration and Utility modules evolving in parallel to support the others.

6.4 Simulation-Based Verification

Each module and the fully integrated firmware were verified using the Wokwi simulator. This allowed rapid iteration on the safety threshold logic and display formatting without the overhead of repeated physical reflashing, and made it straightforward to reproduce specific fault scenarios, such as a simulated over-voltage condition on a single cell, in a controlled and repeatable manner.

6.5 Cloud Integration and Telemetry Testing

The Blynk mobile application was configured with virtual pin bindings corresponding to pack voltage, SOC, SOH, runtime estimate, and fault status. Telemetry transmission was tested for both normal periodic updates and immediate fault notifications, and Wi-Fi reconnection behaviour was verified by temporarily disabling and re-enabling the network connection during operation.

6.6 Test Case Design

A structured set of test cases was designed to exercise both normal and fault-inducing operating conditions.

Test ID	Test Condition	Expected Outcome
TC-01	All cell voltages within normal range	Relay closed; no alarm; normal SOC/SOH display
TC-02	One cell voltage raised above over-voltage threshold	Relay opens; buzzer activates; fault shown on LCD and Blynk
TC-03	One cell voltage lowered below under-voltage threshold	Relay opens; buzzer activates; fault shown on LCD and Blynk
TC-04	Fault condition cleared	Relay re-closes; buzzer deactivates after debounce interval
TC-05	Wi-Fi connection interrupted	Local operation continues; telemetry resumes on reconnection

Table 6.1: Representative Test Case Design

7. RESULTS AND DISCUSSION

7.1 Functional Verification Results

The completed system was evaluated against the test cases defined in Table 6.1, using the Wokwi simulation environment . All core functional requirements were verified successfully, with the system correctly distinguishing between normal and fault operating conditions and responding within the expected timeframe.

Test ID	Environment	Observed Result	Status
TC-01	Wokwi	Relay remained closed; LCD showed Normal status; correct SOC displayed	Pass
TC-02	Wokwi	Relay opened within one control cycle; buzzer activated; over-voltage fault logged	Pass
TC-03	Wokwi	Relay opened within one control cycle; buzzer activated; under-voltage fault logged	Pass
TC-04	Wokwi	Relay re-closed and buzzer silenced after debounce interval	Pass
TC-05	Wokwi	Local protection continued uninterrupted; telemetry resumed after reconnection	Pass

Table 7.1: Summary of Functional Test Results

7.2 Response Time of Protection Mechanism

The time elapsed between the artificial introduction of a fault condition (via potentiometer adjustment) and the corresponding relay disconnection was measured across multiple trials. The observed response time remained consistently close to the 500 millisecond control-loop period,

confirming that the protection logic responds within a single sampling cycle as intended, without noticeable additional software latency.

7.3 SOC and SOH Estimation Behaviour

The SOC estimation output tracked the expected trend as the simulated cell voltages were manually varied across the full operating range, producing a smooth, monotonically increasing percentage as voltage increased from the empty-cell to full-cell reference points. The SOH heuristic indicator remained stable under balanced cell conditions and decreased appropriately when an artificial imbalance was introduced between the four simulated cells, confirming that the imbalance-sensitive component of the estimation logic behaved as designed.

7.4 Telemetry Performance

Telemetry updates to the Blynk dashboard were observed to arrive consistently within the configured two-second update interval under normal Wi-Fi conditions, with immediate fault notifications observed to reach the dashboard within a single additional control cycle of the fault being detected locally. Brief interruptions to the Wi-Fi connection did not affect local protection behaviour, and telemetry resumed automatically once connectivity was restored, without requiring a device restart.

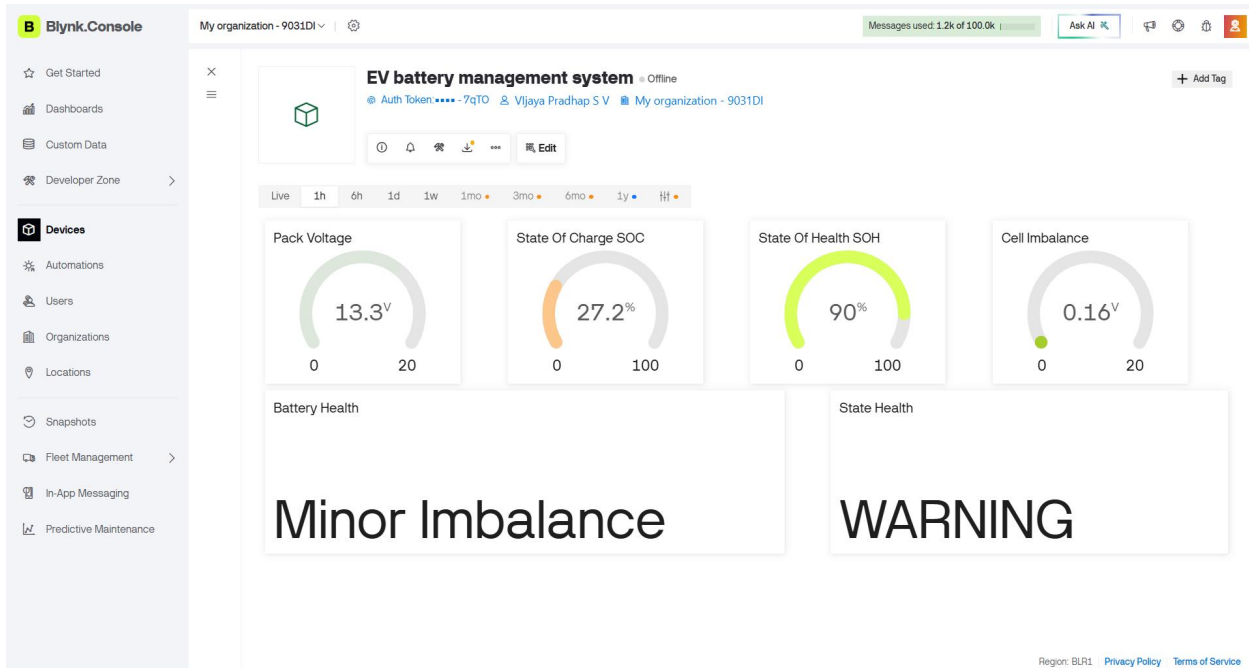


Figure 7.1: Blynk Dashboard Showing Live Pack Voltage, SOC, and Status

7.5 Discussion

The results confirm that the modular firmware architecture described in Chapter 5 successfully delivers the required functional behaviour while keeping the local protection path fast and independent of the state of the telemetry and display subsystems. The use of potentiometers to emulate battery cell behaviour proved effective for repeatable fault-condition testing, though it is acknowledged that this approach does not capture certain real-cell phenomena such as internal resistance rise under load, capacity fade over charge cycles, or genuine thermal behaviour, all of which would need to be addressed before the system could be adapted for use with a physical lithium-ion pack. These limitations provide opportunities for future enhancements.

8. ADVANTAGES & APPLICATIONS

- Low development and prototyping cost, achieved through the use of an inexpensive ESP32 development board and readily available peripheral modules.
- Safe, repeatable testing of fault conditions using potentiometer-based cell emulation, eliminating the risk associated with deliberately stressing real battery cells during development.
- Modular firmware architecture that simplifies maintenance, testing, and future extension of individual subsystems without disrupting the rest of the codebase.
- Combined local and remote observability, providing immediate on-device feedback via the LCD and buzzer alongside remote visibility through the Blynk dashboard.
- Fast, priority-driven protective response, with fault detection and relay disconnection executed ahead of display and telemetry updates within each control cycle.
- Scalable design that can be extended to additional cells, alternative estimation algorithms, or additional sensor inputs with limited changes to the existing module interfaces.
- Integrated Wi-Fi connectivity on the ESP32 removes the need for a separate communication module for cloud telemetry, reducing bill-of-materials complexity.

Although developed and validated in a laboratory/prototyping context using simulated cells, the architecture and techniques demonstrated in this project are broadly applicable to a range of real-world battery-powered systems. Representative application areas are outlined below.

8.1 Electric Mobility

Small-scale electric vehicles such as e-bikes, e-scooters, and light electric utility vehicles require continuous monitoring of multi-cell battery packs to prevent over-discharge during operation and over-charge during recharging, both of which the demonstrated safety logic directly addresses.

8.2 Renewable Energy Storage

Solar and other renewable energy storage installations rely on battery banks whose safe charge and discharge limits must be enforced to protect both the batteries and connected loads; the

relay-based protection and cloud telemetry approach demonstrated here is directly transferable to such installations at a larger scale.

8.3 Unmanned Aerial Vehicles and Robotics

Drones and mobile robots typically use multi-cell lithium-polymer packs that are particularly sensitive to over-discharge; a lightweight, ESP32-based monitoring subsystem of the type demonstrated in this project could be integrated into such platforms to provide in-flight or in-operation voltage monitoring and low-voltage warning.

8.4 Portable and Backup Power Systems

Portable power stations and small uninterruptible power supply (UPS) units benefit from the same combination of local protection and remote status monitoring demonstrated in this project, particularly in unattended or remotely located installations.

8.5 Educational and Research Platforms

The modular, well-documented architecture of this project makes it a suitable teaching platform for demonstrating embedded systems, IoT integration, and battery management concepts in academic laboratory settings.

9. FUTURE SCOPE

While the current implementation successfully meets its defined objectives, several enhancements could extend its capability and bring it closer to a production-grade Battery Management System.

1. Replacing the potentiometer-based cell emulation with real battery cell interfacing through appropriately isolated voltage-sensing and current-sensing front-end circuitry, including a dedicated coulomb counter for higher-accuracy SOC estimation.
2. Implementing active or passive cell balancing circuitry to equalise charge distribution across cells in a real multi-cell pack, extending the Safety Module's scope beyond fault detection into corrective action.
3. Incorporating a dedicated temperature sensor array (e.g. multiple NTC thermistors or digital sensors) positioned at different points on a physical battery pack, rather than a single simulated temperature input.
4. Improving SOC and SOH estimation accuracy through model-based approaches such as extended Kalman filtering or machine-learning-based estimators trained on real charge-discharge cycle data.
5. Migrating the relay-based disconnect mechanism to a solid-state MOSFET-based protection circuit for faster switching, higher reliability, and elimination of mechanical wear.
6. Extending the Telemetry Module to support additional cloud platforms or a self-hosted dashboard, along with historical data logging and trend analysis.
7. Adding over-current protection and short-circuit detection to complement the existing over-voltage, under-voltage, and over-temperature protections.
8. Conducting formal safety and reliability testing, including fault-injection testing and compliance evaluation against relevant battery safety standards, prior to any real-world deployment.

10. CONCLUSION

This report has presented the design, implementation, and validation of an ESP32-based EV Battery Management System developed during the course of an internship programme. The system successfully demonstrates the core functional pillars of a Battery Management System — sensing, estimation, and protection — within a safe, potentiometer-emulated testing environment, while additionally providing local visualisation through a 16×2 I2C LCD and remote monitoring through the Blynk IoT platform.

The modular software architecture, comprising the Battery, Safety, Display, Runtime, Telemetry, Configuration, and Utility modules, proved effective in keeping the codebase organised, testable, and extensible, and enabled systematic verification through the Wokwi simulation environment. Functional testing confirmed that the system reliably detects over-voltage, under-voltage, and fault conditions, responds with prompt relay-based disconnection and buzzer alarm activation, and maintains stable cloud telemetry under normal operating conditions.

Beyond meeting its stated technical objectives, this project provided substantial practical exposure to embedded firmware development using PlatformIO and the Arduino framework, circuit design and simulation-based validation, IoT platform configuration, and structured software version control using Git and GitHub. The experience gained through this internship has strengthened both the theoretical understanding and the hands-on engineering skills necessary for developing embedded IoT systems, and the resulting project provides a solid foundation for further development towards a production-oriented Battery Management System, as outlined in the future scope discussed in Chapter 9.

11. REFERENCES

- [1] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems, 2023.
- [2] Arduino, “Arduino Core for ESP32 – Documentation,” arduino-esp32 Project Documentation.
- [3] PlatformIO, “PlatformIO Documentation – Espressif 32 Platform,” PlatformIO Labs.
- [4] Wokwi, “Wokwi Simulator Documentation – ESP32 and Peripheral Simulation,” Wokwi Online IDE.
- [5] Blynk Technologies, “Blynk IoT Platform Documentation,” Blynk.io Developer Documentation.
- [6] D. Andrea, “Battery Management Systems for Large Lithium-Ion Battery Packs,” Artech House, Boston, 2010.
- [7] V. Pop, H. J. Bergveld, D. Danilov, P. P. L. Regtien, and P. H. L. Notten, “Battery Management Systems: Accurate State-of-Charge Indication for Battery-Powered Applications,” Springer, 2008.
- [8] G. L. Plett, “Battery Management Systems, Volume I: Battery Modeling,” Artech House, Boston, 2015.
- [9] Texas Instruments, “Fundamentals of Battery Fuel Gauging,” Application Report, Texas Instruments.
- [10] IEEE Standards Association, “IEEE Standard for Rechargeable Batteries Used in Portable Applications,” IEEE Std 1625.
- [11] GitHub Repository: “EV Battery Management System – Project Source Code,” [<https://github.com/vj-spec/EV-Battery-Management-System>].